prepare_fx#

https://pytorch.org/docs/stable/generated/torch.ao.quantization.quantize_fx.

Description:

Prepare a model for post training quantization model (*) – torch.nn.Module model qconfig_mapping (*) – QConfigMapping object to configure how a model is quantized, see QConfigMapping for more details example_inputs (*) – Example inputs for forward function of the model, Tuple of positional args (keyword args can be passed as positional args as well) prepare_custom_config (*) – customization configuration for quantization tool. See PrepareCustomConfig for more details

Function Signature

```
class torch.ao.quantization.quantize_fx.prepare_fx(model,
qconfig_mapping, example_inputs, prepare_custom_config=None,
_equalization_config=None, backend_config=None)[source]#

Parameters

Returns

Return type
```

Returns:

GraphModule

Code Examples

```
import torch
from torch.ao.quantization import get_default_qconfig_mapping
from torch.ao.quantization.quantize_fx import prepare_fx

class Submodule(torch.nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.linear = torch.nn.Linear(5, 5)
    def forward(self, x):
        x = self.linear(x)
        return x

class M(torch.nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.linear = torch.nn.Linear(5, 5)
        self.sub = Submodule()

    def forward(self, x):
        x = self.linear(x)
        x = self.sub(x) + x
        return x

# initialize a floating point model
float_model = M().eval()

# define calibration function
```

```
def calibrate(model, data_loader):
    model.eval()
    with torch.no_grad():
        for image, target in data_loader:
            model(image)

# qconfig is the configuration for how we insert observers for a
particular
# operator
# qconfig = get_default_qconfig("fbgemm")
# Example of customizing qconfig:
# qconfig = torch.ao.quantization.QConfig(
#    activation=MinMaxObserver.with_args(dtype=torch.qint8),
#    weight=MinMaxObserver.with_args(dtype=torch.qint8))
# `activation` and `weight` are constructors of observer module

# qconfig_mapping is a collection of quantization
configurations, user can
# set the qconfig for each operator (torch op calls, functional
calls, module calls)
# in the model through qconfig_mapping
# the following call will get the qconfig_mapping that works
best for models
# that target "fbgemm" backend
qconfig_mapping = get_default_qconfig_mapping("fbgemm")

# We can customize qconfig_mapping in different ways.
# e.g. set the global qconfig, which means we will use the same
qconfig for
# all operators in the model, this can be overwritten by other
settings
# qconfig_mapping = QConfigMapping().set_global(qconfig)
# e.g. quantize the linear submodule with a specific qconfig
# qconfig_mapping = QConfigMapping().set_module_name("linear",
qconfig)
# e.g. quantize all nn.Linear modules with a specific qconfig
# qconfig_mapping =
QConfigMapping().set_object_type(torch.nn.Linear, qconfig)
```

```
# for a more complete list, please see the docstring for
:class:`torch.ao.quantization.QConfigMapping`
# argument

# example_inputs is a tuple of inputs, that is used to infer the
type of the
# outputs in the model
# currently it's not used, but please make sure
model(*example_inputs) runs
example_inputs = (torch.randn(1, 3, 224, 224),)

# TODO: add backend_config after we split the backend_config for
fbgemm and qnnpack
# e.g. backend_config = get_default_backend_config("fbgemm")
# `prepare_fx` inserts observers in the model based on
qconfig_mapping and
# backend_config. If the configuration for an operator in
qconfig_mapping
# is supported in the backend_config (meaning it's supported by
the target
# hardware), we'll insert observer modules according to the
qconfig_mapping
# otherwise the configuration in qconfig_mapping will be ignored
#
# Example:
# in qconfig_mapping, user sets linear module to be quantized
with quint8 for
# activation and qint8 for weight:
# qconfig = torch.ao.quantization.QConfig(
#     observer=MinMaxObserver.with_args(dtype=torch.quint8),
#     weight=MinMaxObserver.with-args(dtype=torch.qint8))
# Note: current qconfig api does not support setting output
observer, but
# we may extend this to support these more fine grained control
in the
# future
#
# qconfig_mapping =
```

```
QConfigMapping().set_object_type(torch.nn.Linear, qconfig)
# in backend config, linear module also supports in this
configuration:
# weighted_int8_dtype_config = DTypeConfig(
#   input_dtype=torch.quint8,
#   output_dtype=torch.quint8,
#   weight_dtype=torch.qint8,
#   bias_type=torch.float)

# linear_pattern_config = BackendPatternConfig(torch.nn.Linear)
\
#
.set_observation_type(ObservationType.OUTPUT_USE_DIFFERENT_OBSER
VER_AS_INPUT) \
#    .add_dtype_config(weighted_int8_dtype_config) \
#    ...

# backend_config = BackendConfig().set_backend_patter
```

Detailed Documentation

Documentation

Prepare a model for post training quantization

model (*) – torch.nn.Module model

qconfig_mapping (*) – QConfigMapping object to configure how a model is quantized, see QConfigMapping for more details

example_inputs (*) – Example inputs for forward function of the model, Tuple of positional args (keyword args can be passed as positional args as well)

prepare_custom_config (*) – customization configuration for quantization tool. See PrepareCustomConfig for more details

_equalization_config (*) – config for specifying how to perform equalization on the model

backend_config (*) – config that specifies how operators are quantized in a backend, this includes how the operators are observed, supported fusion patterns, how quantize/dequantize ops are inserted, supported dtypes etc. See BackendConfig for more details

A GraphModule with observer (configured by qconfig_mapping), ready for calibration